
Can This Agent Even Do That?

A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Anonymous Author(s)

Affiliation

Address

email

Abstract

LLM agents are increasingly driven by natural-language specifications, expressed through artifacts such as `SKILL.md` and `AGENT.md` files that describe an agent’s capabilities, tools, and workflows. Agent Skills, for example, package instructions that guide an agent in performing specific tasks [5]. But how can we know whether such specifications are themselves correct? Even if an LLM agent follows a specification exactly, can it actually achieve the goal that the specification is intended to accomplish? Or might the goal be fundamentally unachievable, discovered only after spending substantial time and tokens executing the agent—or even ending in deadlock?

We present a *skill compiler* that formally validates, before runtime execution and without relying on an LLM judge, whether a skill’s *goal is achievable*. The compiler combines capability-guarded reachability from automated planning with deadlock-freedom from **simple multiparty sessions (SMPS)**, decided *directly* over session configurations and a goal-marked global type synthesized from the natural-language specification. Our approach follows the **SMPS** discipline of checking a session directly against a global protocol [1, 2].

The approach begins by using an LLM to extract the content of a natural-language specification and compile it into a formal logical pack consisting of preconditions, effects, and goals, which are then consumed by the inference rules. The checker is specified by a Coq-verified, refutation-sound abstraction theorem.

We further establish subject reduction and session fidelity for the direct discipline. The single-label communication fragment with bystander interleaving is mechanized, while world-changing action interleaving is proved under explicit hypotheses of goal stability and effect commutativity. We provide the syntax, labelled operational semantics, typing and achievability judgments, and the associated metatheory: refutation soundness, tolerance soundness, and capability-monotonicity soundness; operational correspondence; decidability within a static-topology fragment; and an undecidability boundary for an extension with dynamic participant spawning.

1 Introduction

A modern agent is rarely a single model call, but neither is it simply a model. Around the model sits a *harness*: the runtime infrastructure that fixes the execution environment, the tool interface, the context, the orchestration and the verification, and on which reliability depends at least as much as on the base model [3, 4]. A *skill* is one increasingly common way of writing part of that harness down — a natural-language artifact, a folder of instructions the agent discovers and loads on demand [5], and

increasingly drafted by an LLM from a user’s stated goal [6, 7] — that *harnesses* a general-purpose agent toward a particular outcome, naming the tools it may use, prescribing the plan it should follow and, when several agents are involved, fixing who must talk to whom. A skill is behavioural guidance layered over an agent, not a substitute for it: the agent retains its latitude to interpret that guidance at run time. The appeal is that a non-expert can say *what* they want and have the system assemble the *how*. The question the arrangement leaves open — and the question this paper answers — is how anyone knows that the guidance is good enough to achieve what it claims.

Today that answer arrives only by running the skill and reading the wreckage afterwards [4]. Across seven state-of-the-art open-source multi-agent systems, measured failure rates range from 41% to 86.7% [8]; on one of them a specification-level intervention alone — adding a task-objective verification step, with the underlying model held fixed — yields a 15.6% improvement in task success [8, Appendix H], and trace-driven repair of the harness alone lifts task completion by 6.3 to 18.4 points across four agent benchmarks [4]. A substantial share of the defect therefore lies in the guidance rather than in the model. Part of that share is worse than unlucky: the assembled how was *impossible* before the first token was spent, and nothing noticed until execution had already taken actions and, in the multi-agent case, deadlocked halfway.

The failures are structural and recurring. A plan *books a flight and emails a confirmation* although no email tool is in scope (a hallucinated capability). A plan pursues *a flight under \$500* when the only booking tool returns premium fares (an unsatisfiable refinement). A planner waits for a worker’s result while the worker waits for the planner’s answer, each listening for a message the other never sends (a coordination deadlock). In every case the goal was unreachable *a priori*, and a cheap static check could have said so.

This paper asks a deliberately narrow question and answers it with a guarantee: **given an LLM-drafted skill, can its goal be achieved at all?** We are not (here) verifying that every step is safe, nor that the agent behaves correctly on every input. We decide *achievability*, tolerantly — small details, detours, and payload variation must not raise a false alarm — and *soundly for refutation*: when the compiler says a goal is impossible, it is right.

1.1 The idea, and the trust boundary

The architecture has two halves separated by a trust boundary. An LLM performs *compaction*: it reads the prose skill and distills it into a formal *pack* — capabilities annotated with preconditions and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*; the LLM may misread the prose. A deterministic *checker* then decides achievability over the pack, and the checker is mechanically proved sound. The division is what makes the system honest: a faulty compaction can only ever cause a false *achievable* (caught downstream), never a false *impossible* about the pack it received (Figure 1).

Crucially, the semantic gap between *what the user meant* and *what was formalized* is not eliminated — it is *relocated and concentrated* into one inspectable checkpoint at compaction time, rather than scattered, unobserved, across every run. That relocation is the central design move.

1.2 Contributions

1. **A goal-achievability type discipline** (Section 3 and Appendices A–C) that fuses capability-guarded reachability (planning) with deadlock-freedom (multiparty *sessions*) over a goal-marked global type synthesized from intent. Following *SMPS* [1, 2], the declarative discipline types whole session configurations directly; our contribution is to add the guarded world, goal observation, and refutation-sound/achievement-incomplete verification asymmetry for LLM-drafted agent skills.
2. **A mechanized soundness specification** (Appendix D) in Coq: a refutation-sound abstraction theorem parameterized by simulation hypotheses, tolerance soundness (coarsening the abstraction preserves refutation), and capability monotonicity (more tools never make a goal impossible) — all axiom-free — together with an *operational-correspondence* metatheory (subject reduction and session fidelity) for the direct judgment over a labelled transition system. Full bystander interleaving is mechanized for single-label communication; the world-changing action cases are paper proofs under explicit side conditions.

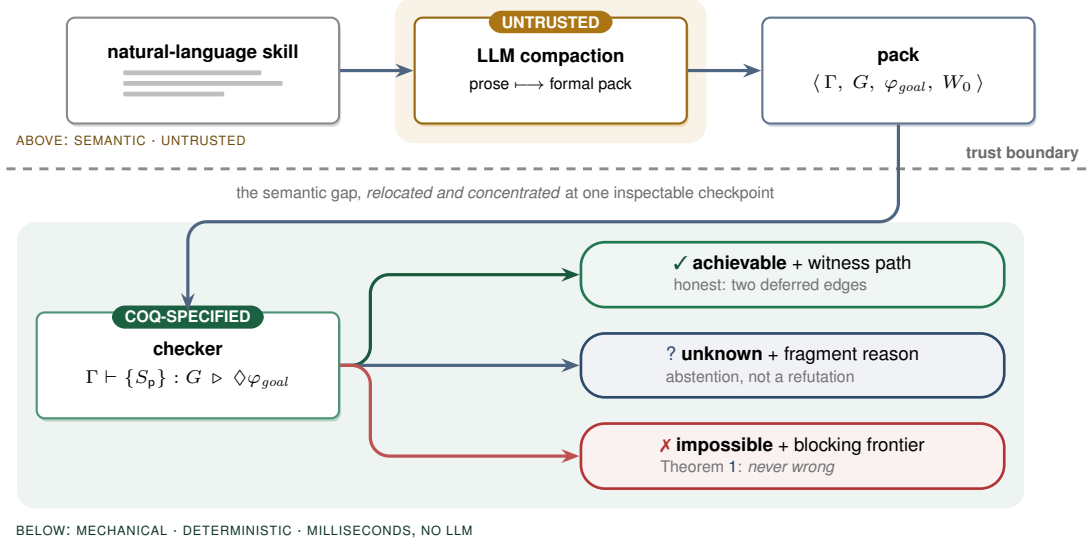


Figure 1: The trust boundary. An untrusted LLM compacts prose into a formal pack; below the line, a deterministic checker governed by a Coq-verified abstraction theorem decides achievability over the pack — the pack is the only artifact that crosses. The verdicts are asymmetric: *impossible* is sound (Theorem 1), *achievable* is honest about its deferred obligations (Section 3.1).

3. **A decidability boundary result** (Appendix D): achievability is decidable in a static-topology fragment; an explicit extension with unbounded participant spawning crosses to undecidability.
4. **An implementation and evaluation** (Sections 4–5): a checker, an LLM-compaction front-end with a schema gate, and a corpus spanning the canonical failure modes. Results match the theory: zero false impossibilities; every false achievability is one of the two deferred residues.

2 Overview by Example

Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The author writes prose; the LLM compacts it. Two compactions differ only in whether an email tool is in scope.

$$\varphi_{goal} = \text{booked} \wedge \text{confirmation_sent}$$

capability	precondition	effect
<i>variant A</i> — no email tool in scope		
search	$\top$	$Add\{\text{searched}\}$
filter	searched	$Add\{\text{filtered}\}$
book	filtered	$Add\{\text{booked}\}$
<i>variant B</i> — variant A, plus:		
email	booked	$Add\{\text{confirmation_sent}\}$

Under STRIPS *frame semantics* — a predicate is false unless an effect makes it true — variant A has no action establishing `confirmation_sent`, so no reachable state satisfies the goal: the checker returns *impossible* and points at the missing establishment. Variant B reaches the goal along `search · filter · book · email`, returned as a witness. The same example is mechanized in Appendix D (the concrete instance `FlightInstance`): variant A is *provably* unachievable, yet a booking is still reachable — the refutation is about the missing tool, not a dead protocol.

Tolerance is essential, and is where a naive reachability check would misfire. Extra status messages, a clarifying detour, or payload variation must not refute the goal. We obtain this from three orthogonal relaxations made precise in Appendices A–C: *existential may-reachability* (a single good path

107 suffices), *session subtyping* (interface slack in the safe direction, folded directly into the single
 108 conformance rule rather than a separate relation), and *goal-relevant abstraction* (payload detail
 109 collapsed before checking).

110 3 Verification Model and Signals

111 **Declared pack.** The checker consumes $P = \langle \Gamma, G, \varphi_{goal}, W_0 \rangle$: capability declarations Γ with
 112 guards and STRIPS-style effects; a goal-marked global protocol G (formally, a *global type*; Ap-
 113 pendix A); a logical goal φ_{goal} ; and an initial abstract world W_0 . The LLM that compacts prose into
 114 P is outside the trusted core. A deterministic schema gate first enforces the decidable fragment: finite
 115 static roles, guarded recursion, finite-domain booleans, and quantifier-free linear integer arithmetic.
 116 The formal syntax and labelled semantics are given in Appendices A–B.

117 **Heterogeneous verification signals.** Achievability is not reduced to a learned score. The judgment
 118 composes signals that live on different substrates:

Signal	Formal role	Evidence returned
tool availability and effects coordination structure	capability guards; world transition direct global conformance	missing capability or witness action offending handoff or conformant proto- col
goal and cost refinements	logical reachability; QF-LIA	unsatisfied conjunct, blocked guard, or model
human intent review runtime payload checks	validates prose-to-pack fidelity deferred monitor obligation	accepted or revised declared goal blame-bearing execution event

120 The first three signals are composed before execution. The latter two are explicit obligations rather
 121 than assumptions silently folded into the static verdict.

122 **Decision procedure.** After the schema gate, the checker (i) rejects references to undeclared capa-
 123 bilities and goal atoms with no establisher; (ii) checks declared role behaviors against G , requiring
 124 exact sender choices while allowing receiver-side branch supersets; and (iii) explores existential pro-
 125 tocol/world successors from W_0 , using Z3 for guards and refinements. An unsatisfied goal marker is
 126 a blocking checkpoint. Solver timeouts and packs outside the static fragment cannot justify refutation.
 127 The interface is therefore tri-valued:

$\text{IMPOSSIBLE}(F)$	a sound blocking frontier F ,
$\text{ACHIEVABLE}(\pi, O)$	a witness path π and deferred obligations O ,
$\text{UNKNOWN}(r)$	an abstention with fragment reason r .

128 Verdicts also carry the pack digest, checker semantics, and artifact version so that feedback remains
 129 attributable to the exact declaration checked.

130 **Guarantees.** Let abs map concrete protocol/world configurations to abstract configurations, and
 131 assume step and goal simulation for the declared pack (Appendix D). If no abstract run reaches
 132 φ_{goal} , no concrete run represented by that pack reaches the concrete goal (Theorem 1). Coarsening
 133 α and adding capabilities cannot create a false refutation (Theorems 2–3). Direct conformance has
 134 subject-reduction and session-fidelity results (Theorems 4–5); the Coq artifact covers single-label
 135 communication with bystander interleaving, while the paper states explicit stability and commutativity
 136 assumptions for world-changing interleavings. The static fragment is decidable; unbounded dynamic
 137 spawning is an explicit undecidable extension, for which the implementation abstains. Full rules,
 138 assumptions, proofs, and mechanization scope appear in Appendix D.

139 3.1 Structured feedback and human oversight

140 The blocking frontier and witness are inspectable, heterogeneous feedback rather than a scalar reward.
 141 A bounded repair adapter may ask the untrusted LLM to fix a non-projectable protocol, but it may
 142 neither invent capabilities nor weaken the goal, and every revision is rechecked deterministically.
 143 This realizes a small agent-verifies-agent loop while keeping the human checkpoint at the irreducible
 144 intent-fidelity stage. Automated prompt or scaffold search is an application of these verdicts, not an
 145 empirical contribution claimed here.

4 Implementation

The executable decision path is approximately 1.45K lines of Python, including the checker, pack gate, formula layer, and session adapter. The compaction front-end issues the artifact prompt to an LLM and passes the result through a deterministic *schema gate* before the checker sees it, so malformed packs are rejected without crashing the trusted core. The Coq artifact contains three developments and two assumption-audit harnesses: reachability soundness core (Theorems 1–3 plus `FlightInstance`, ~230 lines); the direct-typing core (the head-move action safety plus `HandoffInstance`, ~310 lines); and the operational-correspondence core (Theorems 4–5 for the single-label communication fragment with full interleaving, ~250 lines); all three are checked in CI under pinned Coq 8.18.0 with no axioms, verified by `Print Assumptions`. The checker implements the corresponding abstractions from Appendices A–C: STRIPS frame semantics for the world, existential search over the protocol control structure, and Z3 for guard/goal satisfiability and arithmetic refinements. On success it returns the witness path; on refutation, the blocking frontier (missing capability, unsatisfiable guard, unestablished goal conjunct, or non-projectable handoff). The optional LLM front-end may use a `NON_PROJECTABLE` counterexample for one bounded, structure-only repair round; it may not invent capabilities or weaken the goal, and the deterministic checker remains the final referee. This is a small reflective-verification loop, not a general scaffold-search result.

The conformance step ($G \vdash (\mathbb{M}; W)$, Section C.2) is **implemented by means of a simple type inference algorithm since all typing rules are structural in $(\mathbb{M}; W)$. At each step we freely choose either one or two participant processes to reduce or a goal satisfied by the world obtaining the set of sessions which must be typed as premises of the typing rule.**

The adapter also discharges the participant side conditions of T-COMM/T-ACT/T-GOAL: it computes $\text{prt}(G)$ and compares it with the roles the pack declares behaviour for. One direction refutes — a declared role outside $\text{prt}(G)$ has contract `End`, which nothing non-trivial refines. The other cannot: a participant the pack declares nothing for offers no behaviour to refute, yet the judgment of Section C.2 ranges over a whole session with $\text{prt}(G) = \text{prt}(\mathbb{M})$, so the checker returns those roles alongside the verdict rather than leaving the assumption implicit.

5 Evaluation

We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning, retry-forever loops, coordination deadlock, refinement violations — with achievable controls (including detours and informed branches) and two deliberately *spurious* cases exercising the deferred edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*. Evaluation is deterministic and CPU-only: it requires no training or live model inference, completes the 15-case matrix in under one second on a commodity laptop, and gives each individual Z3 query a 10-second timeout.

	predicted achievable	predicted impossible
truly achievable	TP = 6	FN = 0
truly impossible	FP = 2	TN = 7

Two claims, both confirmed. **Soundness:** FN = 0 — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall 6/6 = 100%). **Incompleteness, contained:** FP = 2, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). No structural failure was missed in this proof-of-concept corpus; this is evidence of coverage, not a proof that the two residues exhaust all empirical failure modes. Each refutation reason fires on its matching mode:

Failure mode (source)	Verdict reason
hallucinated planning — invokes a non-existent tool	MISSING_CAPABILITY
hallucinated planning — no establisher for a goal conjunct	GOAL_UNSAT
retry-forever loop — guard never satisfiable	BLOCKED_GUARD
coordination deadlock — unobserved choice / bad handoff	NON_PROJECTABLE
refinement violation — budget unsatisfiable on every run	GOAL_UNSAT

What the check costs. With no model in the trusted path, the checker and the deterministic front-end spend *zero* tokens; only compaction spends any, once per skill *version* and linearly in the skill’s length, against a run it avoids that recurs per *invocation* and is quadratic in its turns. Under an explicit, conservative model of that run (Appendix E), compacting the 15-spec corpus costs 12.2K tokens and avoids $\sim 1.07\text{M}$ per round of invocations — $\sim 87\times$ leverage, unbounded deterministically — so break-even falls inside the first prevented run in every failure mode, while a skill that is *not* broken pays 2.9% of one successful run for the check, or nothing.

A separate six-case fragment suite exercises behavior absent from the headline matrix: tail-recursive retry and non-terminating control, dynamic spawning, protocol-independent refutation despite spawning, and conformant versus non-conformant declared role behavior. It yields two ACHIEVABLE, three IMPOSSIBLE, and one UNKNOWN. The abstention is reported separately rather than counted as a false negative, because UNKNOWN makes no refutation claim. Together the suites contain 21 declared packs; the 15-case matrix remains the only binary ground-truth matrix.

6 Related Work

Prior-art note. The claims below reflect the authors’ survey at submission time; the fusion’s novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

Multiparty session types [12, 15, 9] supply deadlock-freedom via projection and subtyping [13, 10]; our calculus and directed-choice global types follow the modern synchronous formulation of the Yoshida line [9–11]. We adopt the synthetic line [1, 2]: Section C.2 checks conformance directly against a whole session configuration rather than projected local types. Our direct rule corresponds to the conservative, plain-merge condition that bystanders behave uniformly across an unobserved choice; it does not subsume every protocol accepted by modern full merge. The new contribution is the world and goal layer: capability guards, effects, goal observation, and the asymmetric refutation guarantee are not modelled by classical MPST or new SMPS. *Automated planning* [16] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an goal-marked global type synthesized by an LLM. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [17] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; its runtime-planning extension also establishes that LLM synthesis of a coordination protocol is itself prior art. *VeriGuard* [18] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [19] provide probabilistic, runtime-enforced compliance and composition conditions for multi-agent chains, whereas our verdict is static and sound for refutation relative to the pack.

Skill-bundle scanners and verified-skill catalogs are complementary. NVIDIA’s SkillSpector [20] and Verified Agent Skills pipeline [21] address the same deployment object—portable agent skills—but ask a different question. SkillSpector is a pre-publication security control: it normalizes a skill bundle and runs deterministic scanners such as static pattern detectors, AST and taint analyzers, manifest-consistency checks, metadata-poisoning checks, supply-chain checks, and optional LLM-backed semantic analyzers. This is valuable admission control for skills, but it is not a compiler or type discipline in the sense used here. As published, SkillSpector does not aim to provide an operational semantics of goal-marked skills, a goal-reachability decision, or a refutation-soundness theorem. A SkillSpector-like pass belongs at the boundary of the compiler: before or alongside LLM compaction, it can vet the incoming SKILL.md, code, dependencies, and MCP metadata; after compaction, it can help justify the honesty of declared capabilities and least-privilege metadata. The type checker then consumes the sanitized formal pack and proves the narrower claim that the declared goal is not structurally impossible.

None of the above frames the static check as goal achievability over a declared world and goal-marked protocol, nor gives the same refutation-sound/achievement-incomplete asymmetry.

7 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared Γ and S . If the prose lies (a “read-only” tool that writes), the checker verifies a fiction; honest-declaration is a separate

obligation, of which this makes only the interaction slice decidable. A practical compiler should therefore include a skill-bundle security pass—for example a SkillSpector-like scanner—to inspect SKILL.md, scripts, dependencies, manifests, and permission metadata before the formal pack is trusted by the achievability checker.

- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Proposition 2); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse α silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers the generic refutation-sound abstraction theorem and concrete examples, head-move action safety, and subject reduction/session fidelity with full bystander interleaving for the single-label communication model, all axiom-free. The executable checker is schema-gated and tested against that specification, but the STEP-SIM/GOAL-SIM instantiation for its exact symbolic state is not mechanized. The Coq correspondence model also does not yet combine observable goal labels, participant matching, multi-branch choice, and world-changing interleaving in one development. Still open are that faithful mechanization; a head-normalization/permutation bridge from interleaved session runs to the head-ordered reachability search; equivalence between the declarative judgment and the projection-based adapter; and a concrete-session realization theorem for abstract witnesses. The 21-pack evaluation remains a proof of concept.

Broader impact. Pre-execution refutation reduces wasted computation (Appendix E) and prevents structurally impossible agent plans from reaching deployment. The same formal verdict can, however, create false confidence when an inaccurate pack omits a harmful tool effect or weakens the user’s actual goal. We therefore expose the declared pack, witness or blocking frontier, and deferred obligations rather than presenting a single trust score. Intent review and runtime payload monitoring remain necessary before the method is used in consequential settings; this work does not evaluate fairness, privacy, or deployment safety.

8 Conclusion

An LLM can draft an agent’s plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, relative to a declared pack: combine planning’s capability-guarded reachability with a synthetic, directly-typed global protocol; treat LLM compaction as untrusted; and use a refutation-sound abstraction so an *impossible* verdict is never wrong about that declared model. *Achievable* remains honest about payload and intent fidelity, while *Unknown* abstains outside static topology. This yields a structured, inspectable verification signal for agent skills and a path toward, rather than a completed proof of, stronger per-step safety.

References

- [1] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de’Liguoro (2024): *Un-projectable Global Types for Multiparty Sessions*. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella & Matteo Rossi, editors: *PPDP*, ACM Press, pp. 15:1–15:13,
- [2] Francesco Dagnino, Paola Giannini & Mariangiola Dezani-Ciancaglini (2023): *Deconfined Global Types for Asynchronous Sessions*. *Logical Methods in Computer Science* 19(1), pp. 1–41,
- [3] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, X. Liao, Y. Ji, J. Wang, J. Gu, Y. Ge, W. Xu, X. Fang, X. Xu, T. Zhao, Y. Kim, T. Wang, J. Hamm, S. Krishnaswamy, J. Huan, C. Reddy. Agent Harness Engineering: A Survey. 2026.
- [4] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, Q. Wang. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324, 2026.
- [5] Agent Skills Standard. Specification. <https://github.com/agentskills/agentskills/blob/main/docs/specification.mdx>, 2025.
- [6] S. Hu, C. Lu, J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.

- [7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.
- [8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.
- [9] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT*, LNCS 11969, 2020.
- [10] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.
- [11] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.
- [12] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.
- [13] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.
- [14] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.
- [15] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.
- [16] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.
- [17] B. Bollig, M. Függer, T. Nowak. Provable Coordination for LLM Agents via Message Sequence Charts. *ISoLA*, 2026. arXiv:2604.17612.
- [18] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025.
- [19] V. P. Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.
- [20] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.
- [21] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 19 May 2026.

A Syntax

We fix disjoint sets of predicates $p \in \text{Pred}$ (boolean), numeric variables $x \in \text{Var}$, roles $p, q, r \in \mathcal{R}$, capability names $a \in \text{Cap}$, and message labels $\ell \in \text{Lab}$. Following the modern presentation of multiparty sessions [1, 2] we proceed bottom-up: state and capabilities (Sections A.1–A.2), then processes — the programs skills execute as — (Section A.3), then the global type they are checked against directly (Section A.4). In each grammar, “ $::=$ ” lists the possible forms of an expression. In each inference rule below, the statement beneath the line follows when every premise above the line holds. Rule names identify the corresponding checker obligation; they are not additional assumptions.

A.1 State logic

$$\begin{aligned} e &::= x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi &::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{aligned}$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic with uninterpreted boolean predicates — a decidable fragment for which entailment $\models$ and satisfiability are discharged by a satisfiability-modulo-theories (SMT) solver.

330 A.2 Worlds and capabilities

331 A world $W = \langle B, N \rangle$ assigns truth values to predicates $B : \text{Pred} \rightarrow \mathbb{B}$ and numeric variables
332 $N : \text{Var} \rightarrow \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\begin{aligned} \Gamma(a) &= \langle \text{pre}_a, \text{eff}_a \rangle, & \text{pre}_a &\in \mathcal{L}, \\ \text{eff}_a &= \langle \text{Add}_a \subseteq \text{Pred}, \text{Del}_a \subseteq \text{Pred}, \text{Asg}_a : \text{Var} \rightarrow e, \text{Nd}_a : \text{Var} \rightarrow \mathcal{L} \rangle \\ 333 \quad a \notin \text{dom}(\Gamma) &\iff \text{the agent does not possess tool } a \quad (\text{no hallucinated tools}). \end{aligned}$$

334 *Add/Del* are the STRIPS add- and delete-lists; *Asg* gives deterministic numeric updates $x := e$; *Nd*
335 gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to model
336 post-conditions such as “*books a fare under 500*”).

337 A.3 Processes and multiparty sessions

338 Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous
339 multiparty session calculus in its modern coinductive **presentation** [1, 2]: **processes coinductively**
340 **defined are possibly infinite regular trees (finitely many distinct subtrees)**, and labels are goal-relevant
341 tokens, $\ell = \alpha(m)$ for concrete messages m — the tolerance dial α is a parameter of the label
342 alphabet.

$$P ::=^{\text{coind}} \mathbf{0} \mid \mathbf{p}!\{\ell_i.P_i\}_{i \in I} \mid \mathbf{p}?\{\ell_i.P_i\}_{i \in I} \mid a.P \quad \mathbb{M} = \mathbf{p}_1[P_1] \parallel \cdots \parallel \mathbf{p}_n[P_n]$$

343 with I finite and non-empty, labels ℓ_i pairwise distinct, and **role** names pairwise distinct. The output
344 $\mathbf{p}!\{\ell_i.P_i\}$ *internally* chooses a label and sends it to $\mathbf{p}$; the input $\mathbf{p}?\{\ell_i.P_i\}$ offers an *external* choice;
345 $a.P$ invokes capability a — the only construct that changes the world. A skill S_p is a process: the
346 behaviour declared for role $\mathbf{p}$. **why do you need skills? they are only used in C.3 and in a code where**
347 **I think is would be better to use processes**

348 A.4 Global types

349 The single communication construct of a global type is a *directed* labelled choice — selection by $\mathbf{p}$
350 and branching at $\mathbf{q}$ in one construct. (A partnerless “ $\mathbf{p}$ selects” admits no coherent view for the roles
351 that did not choose; directed choice is the standard MPST construct [12, 9].)

$$G ::=^{\text{coind}} \mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I} \mid a @ \mathbf{p}.G \mid \check{\varphi}.G \mid \text{End}$$

352 subject to $\mathbf{p} \neq \mathbf{q}$ and the same regularity and label conditions as processes. This is the *only* type
353 in the discipline: Section C.2 types a whole session directly against G — there is no local-type
354 grammar, no projection function, and no separate subtyping relation. Goal markers $\check{\varphi}$ live *only*
355 in G : achievability is decided on the global type (Rule ACH in C.3), and markers are consumed at
356 typing time by T-GOAL without ever burdening a process. Only $a @ \mathbf{p}$ changes the world; messages
357 carry coordination information only.

358 B Operational Semantics

359 B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because *Nd* is nondeterministic,
 $\langle W \rangle a \langle W' \rangle$ may relate one world to many. The relation is implicitly indexed by Γ : the notation
 $\text{pre}_a, \text{eff}_a$ is defined only when $\Gamma(a) = \langle \text{pre}_a, \text{eff}_a \rangle$, so every firing presupposes $a \in \text{dom}(\Gamma)$.

$$\begin{aligned} &\text{WORLD-ACT} \\ &\frac{W \models \text{pre}_a \quad W' \in \text{apply}(\text{eff}_a, W)}{\langle W \rangle a \langle W' \rangle} \\ 360 \quad &\text{apply}(\text{eff}_a, \langle B, N \rangle) = \langle B', N' \rangle, \quad B' = (B \cup \text{Add}_a) \setminus \text{Del}_a \\ 361 \quad &N'(x) = \begin{cases} \llbracket \text{Asg}_a(x) \rrbracket_N & \text{if } x \in \text{dom}(\text{Asg}_a) \\ v \text{ with } \langle B, N[x \mapsto v] \rangle \models \text{Nd}_a(x) & \text{if } x \in \text{dom}(\text{Nd}_a) \\ N(x) & \text{otherwise (frame)} \end{cases} \end{aligned}$$

362 **Mariangiola: where is $\llbracket \text{Asg}_a(x) \rrbracket_N$ defined?**

363 B.2 A labelled semantics for sessions and global types

364 Only capabilities touch the world, so sessions reduce as configurations $(\mathbb{M}; W)$; communication is
 365 synchronous [10]. Each transition carries a label Λ recording its *participants*, so that independent
 366 moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the
 367 *participants* $\text{prt}(\cdot)$ of a global type, a label, or a session, and the *capabilities* $\text{cap}(\cdot)$ (the set of moves
 368 a global type can perform). On global types,

$$\begin{aligned} \text{prt}(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}) &= \{p, q\} \cup \bigcup_{i \in I} \text{prt}(G_i) & \text{prt}(\check{\varphi}.G) &= \text{prt}(G) \\ \text{prt}(a@p.G) &= \{p\} \cup \text{prt}(G) & \text{prt}(\text{End}) &= \emptyset, \end{aligned}$$

369 on labels $\text{prt}(p \ell q) = \{p, q\}$, $\text{prt}(a@p) = \{p\}$, $\text{prt}(\check{\varphi}) = \emptyset$, and on sessions $\text{prt}(\mathbb{M}) = \{p \mid \mathbb{M} \equiv$
 370 $p[P] \ \& \ P \neq \mathbf{0}\}$ (the roles with a residual behaviour). The capabilities of a global type are

$$\begin{aligned} \text{cap}(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}) &= \{p \ell_i q \mid i \in I\} \cup \bigcup_{i \in I} \text{cap}(G_i) \\ \text{cap}(a@p.G) &= \{a@p\} \cup \text{cap}(G) & \text{cap}(\check{\varphi}.G) &= \{\check{\varphi}\} \cup \text{cap}(G) & \text{cap}(\text{End}) &= \emptyset. \end{aligned}$$

371 The capability names used by a protocol are

$$\text{caps}(G) = \{a \mid \exists p. a@p \in \text{cap}(G)\}.$$

The LTS of sessions is defined by

S-COMM

$$\frac{k \in I \subseteq J}{(p[q!\{\ell_i.P_i\}_{i \in I}] \parallel q[p?\{\ell_j.Q_j\}_{j \in J}] \parallel \mathbb{M}; W) \xrightarrow{p \ell_k q} (p[P_k] \parallel q[Q_k] \parallel \mathbb{M}; W)}$$

S-ACT

$$\frac{\langle W \rangle a \langle W' \rangle}{(p[a.P] \parallel \mathbb{M}; W) \xrightarrow{a@p} (p[P] \parallel \mathbb{M}; W')}$$

S-GOAL

$$\frac{W \models \varphi}{(\mathbb{M}; W) \xrightarrow{\check{\varphi}} (\mathbb{M}; W)}$$

372 where $\parallel$ is commutative, associative and $p[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$. A non-terminated configuration with
 373 no successor is *stuck*; the deadlocking planner/worker handoff of Section 1 is the stuck two-role
 374 configuration in which both processes begin with an input. A goal marker is *not* carried by any
 375 process; instead S-GOAL lets a configuration *observe* that its world already entails one of its declared
 376 goals φ , emitting the label $\check{\varphi}$ without altering $\mathbb{M}$ or W . (Each session carries a set of goal formulas,
 377 and φ ranges over that set — not over every formula the world happens to satisfy — so that each
 378 observation has a matching marker in the protocol.) This observable goal step is matched at the
 379 type level by G-GOAL-E below, so that $\check{\varphi}$ is a genuine label Λ of both transition systems and the
 380 operational correspondence of Section D.3 ranges over it uniformly.

The checker relates a session to its protocol through a *labelled* transition system on global types,
 $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, built from *head* rules that consume the leading construct and *interleaving*
 rules that let a move by participants *disjoint* from the head commute inward:

G-COMM-E

$$\frac{k \in I}{(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{p \ell_k q} (G_k, W)}$$

G-ACT-E

$$\frac{\langle W \rangle a \langle W' \rangle}{(a@p.G, W) \rightsquigarrow_{a@p} (G, W')}$$

G-GOAL-E

$$\frac{W \models \varphi}{(\check{\varphi}.G, W) \rightsquigarrow_{\check{\varphi}} (G, W)}$$

G-COMM-I

$$\frac{(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W') \quad \Lambda \in \text{cap}(G_i) \ \forall i \in I \quad \text{prt}(\Lambda) \cap \{p, q\} = \emptyset}{(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{\Lambda} (p \rightarrow q : \{\ell_i.G'_i\}_{i \in I}, W')}$$

G-ACT-I

$$\frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G) \quad \text{prt}(\Lambda) \cap \{p\} = \emptyset}{(a@p.G, W) \rightsquigarrow_{\Lambda} (a@p.G', W')}$$

G-GOAL-I

$$\frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G)}{(\check{\varphi}.G, W) \rightsquigarrow_{\Lambda} (\check{\varphi}.G', W')}$$

381 G-GOAL-E discharges a goal marker when $W \models \varphi$, emitting the observable label $\checkmark \varphi$ that matches
 382 S-GOAL. In the head-only reduction used for achievability, an unsatisfied marker is a checkpoint and
 383 blocks the path. In the full labelled correspondence, G-GOAL-I lets a continuation step commute
 384 inward under a still-pending marker. When that step changes the world, the correspondence theorem
 385 uses the goal-stability hypothesis stated in Section D.3; removing that hypothesis is future work. The
 386 extraction (head) rules G-COMM-E/G-ACT-E/G-GOAL-E give the Λ -erased reduction $(G, W) \rightsquigarrow$
 387 (G', W') that the achievability judgment below uses; the interleaving rules matter only for the
 388 operational correspondence of Section D.3, and their $\Lambda \in \text{cap}(G)$ premise confines a commuting
 389 move to a capability the residual protocol actually offers. G-COMM-E is legitimate because choice
 390 is directed — the branch is a communication both endpoints observe, and Section C.2 checks that
 391 every third party can follow. A well-formed pack uses its declared goal φ_{goal} at each goal marker. A
 392 configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world
 393 entails that declared goal:

$$394 \quad \begin{aligned} & \text{goal}_{\varphi_{\text{goal}}}(\checkmark \varphi_{\text{goal}}.G, W) \iff W \models \varphi_{\text{goal}}, \quad \text{goal}_{\varphi_{\text{goal}}}(\text{End}, W) \iff W \models \varphi_{\text{goal}}, \\ & \Gamma; G \models \Diamond_{\text{init}} \varphi_{\text{goal}} \iff \exists W_0 \models \text{init}. \exists (G', W'). (G, W_0) \rightsquigarrow^* (G', W') \wedge \text{goal}_{\varphi_{\text{goal}}}(G', W'). \end{aligned}$$

395 The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

396 B.3 Realizability and subtyping, without projection

397 The classical route to ruling out a deadlocking handoff is *projection*: compute each role’s local
 398 type $G|r$ by structural recursion on G , taking the *merge* of a choice’s branches at every role that
 399 neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined
 400 everywhere — undefinedness of the merge is the static signature of a role forced to behave differently
 401 in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation
 402 on local types, letting a role’s actual behaviour offer more external choices and fewer internal ones
 403 than its projected contract demands [13, 10, 11].

404 Both devices exist to answer questions about the *network as a whole* — “can every role tell the
 405 branches apart it needs to?”, “may this implementation stand in for that contract?” — while type-
 406 checking only one role’s syntax at a time. Section C.2 answers the same two questions with a single
 407 judgment, *typing the whole configuration directly against G*: a choice node’s rule quantifies over
 408 every branch the protocol declares and requires the *same* residual configuration to check against each
 409 one. This is the conservative plain-merge condition, obtained without ever computing $\sqcap$ or a local
 410 type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol’s labels)
 411 is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same
 412 three independent knobs from Section 2 — the existential $\Diamond$ (Section B.2), subtyping (now inside
 413 T-COMM, Section C.2), and the granularity of α — “flexible, tolerant of small details” is still the
 414 profile $\langle \text{coarse } \alpha, \Diamond, \text{linear } \mathcal{L} \rangle$; a later safety layer flips the same machinery to the universal $\Box$ over
 415 permission predicates.

416 C The Achievability Type System

417 Achievability now factors into three premises, each separately decidable, combined by the top rule
 418 ACH: no hallucinated capability, direct conformance of the declared skills to G , and reachability of
 419 the goal.

420 C.1 Capability and goal typing

$$421 \quad \begin{array}{c} \text{T-EFF} \\ \Gamma(a) = \langle \text{pre}, \text{eff} \rangle \quad \varphi \models \text{pre} \\ \hline \Gamma \vdash \{\varphi\} a \{\text{sp}(\varphi, \text{eff})\} \end{array} \quad \begin{array}{c} \text{T-MISS} \\ a \notin \text{dom}(\Gamma) \\ \hline \Gamma \vdash a \triangleright \mathbf{X} \text{ (MISSING_CAPABILITY)} \end{array}$$

422 $\text{sp}(\varphi, \text{eff})$ is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning:
 423 the plan names a tool the context does not grant, and achievability is refuted without any search.

423 C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash$
 $(\mathbb{M}; W)$ — read “ G types session $\mathbb{M}$ starting from world W .” There is no local type, no projection,

and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\begin{array}{c}
\text{T-END} \\
\hline
\text{End} \vdash (p[0]; W)
\end{array}
\quad
\begin{array}{c}
\text{T-ACT} \\
\hline
\frac{G \vdash (p[P] \parallel \mathbb{M}; W') \quad \langle W \rangle a \langle W' \rangle \quad \text{prt}(G) \cup \{p\} = \text{prt}(\mathbb{M}) \cup \{p\}}{a@p. G \vdash (p[a.P] \parallel \mathbb{M}; W)}
\end{array}$$

$$\begin{array}{c}
\text{T-GOAL} \\
\hline
\frac{G \vdash (\mathbb{M}; W) \quad W \models \varphi \quad \text{prt}(G) = \text{prt}(\mathbb{M})}{\checkmark \varphi. G \vdash (\mathbb{M}; W)}
\end{array}$$

$$\begin{array}{c}
\text{T-COMM} \\
\hline
\frac{G_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}; W) \quad \forall i \in I \subseteq J \quad \text{prt}(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}) = \text{prt}(\mathbb{M}) \cup \{p, q\}}{p \rightarrow q : \{\ell_i.G_i\}_{i \in I} \vdash (p[q!\{\ell_i.P_i\}_{i \in I}] \parallel q[p?\{\ell_j.Q_j\}_{j \in J}] \parallel \mathbb{M}; W)}
\end{array}$$

424 T-ACT pairs type and world in lockstep with WORLD-ACT: the unconsumed type $a@p.G$ sits with
 425 the *pre*-effect world W and the residual G with the *post*-effect world W' — the same convention
 426 as G-ACT-E (Section B.2). In T-COMM the sender p offers exactly the protocol's branch set I ,
 427 while the receiver q may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (safe
 428 interface slack at the receiver). Requiring *every* protocol branch ℓ_i to be checked against the *same*
 429 bystanders $\mathbb{M}$ is the plain-merge form of the unobserved-choice condition, so a role forced to behave
 430 differently across branches it cannot distinguish makes no derivation possible — the rule fails closed,
 431 and deadlock-freedom falls out of T-COMM itself with no merge to compute (Theorem 4). Each
 432 rule additionally carries a *participant-matching* side condition tying the protocol's participants to
 433 the session's: $\text{prt}(p \rightarrow q : \{\ell_i.G_i\}) = \text{prt}(\mathbb{M}) \cup \{p, q\}$ in T-COMM, $\text{prt}(G) \cup \{p\} = \text{prt}(\mathbb{M}) \cup \{p\}$
 434 in T-ACT, and $\text{prt}(G) = \text{prt}(\mathbb{M})$ in T-GOAL. These make the invariant “the roles the protocol still
 435 mentions are exactly the roles still running” hold at every node, ruling out a session that carries a stray
 436 participant the protocol has forgotten (or vice versa) — the well-formedness that lets the interleaving
 437 cases of Section D.3 match a bystander move on the session to a $\text{cap}(\cdot)$ -labelled move on the type.
 438 T-END closes the discipline at the terminus: the terminated protocol End types a single idle role $p[0]$
 439 — and, through the congruence $p[0] \parallel \mathbb{M} \equiv \mathbb{M}$, any session all of whose roles have terminated —
 440 the base case of the participant invariant, since $\text{prt}(\text{End}) = \emptyset$ and an idle role contributes nothing to
 441 $\text{prt}(\mathbb{M})$. T-ACT is derivable only when $\langle W \rangle a \langle W' \rangle$ holds, which itself presupposes $a \in \text{dom}(\Gamma)$:
 442 this is where hallucinated capabilities die, with T-MISS as the diagnostic.

443 C.3 The achievability judgment

$$\begin{array}{c}
\text{ACH} \\
\hline
\frac{\exists W_0 \models \text{init}. G \vdash (p_1[S_{p_1}] \parallel \dots \parallel p_n[S_{p_n}]; W_0) \wedge \Gamma; (G, W_0) \models \Diamond \varphi_{\text{goal}}}{\Gamma \vdash \{S_p\}_p : G \triangleright \Diamond \varphi_{\text{goal}}}
\end{array}$$

444 **Mariangiola: who is p ?** Read top-to-bottom: no hallucinated tools; some plausible initial world exists
 445 from which the declared skills directly conform to the protocol (deadlock-free, every capability call
 446 guarded) *and* that same world reaches the goal. The reachability half is unchanged from Section B.2
 447 and decided on Γ, G alone, before any S_p is known — exactly what lets the checker run on the
 448 LLM-compacted pack the moment it exists; conformance is the additional, separately decidable
 449 check once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is
 450 already inside T-COMM.

451 C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-COMM/T-ACT/ T-GOAL, it never mentions a session $\mathbb{M}$, which is what

makes it decidable on the pack alone.

$$\begin{array}{c}
\text{REACH-GOAL} \\
\frac{W \models \varphi_{\text{goal}} \quad G = \text{End} \vee \exists G'. G = \checkmark \varphi_{\text{goal}}. G'}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \\
\\
\begin{array}{cc}
\text{REACH-STEP} & \text{REACH-OR} \\
\frac{(G, W) \rightsquigarrow (G', W') \quad \Gamma; (G', W') \models \Diamond \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} & \frac{\exists k \in I. \Gamma; (G_k, W) \models \Diamond \varphi_{\text{goal}}}{\Gamma; (\text{p} \rightarrow \text{q}; \{\ell_i. G_i\}_{i \in I}, W) \models \Diamond \varphi_{\text{goal}}}
\end{array}
\end{array}$$

452 D Metatheory

453 We separate what is *mechanized in Coq* (Theorems 1–3; Theorems 4–5 for the single-label com-
 454 munication fragment with full interleaving; the head-move action case; and the concrete instances,
 455 all axiom-free) from what is *proved on paper* (the action-interleaving cases, decidability, and the
 456 undecidability boundary).

457 D.1 Soundness of the abstraction

458 Let $\mathcal{C}$ be the concrete configuration space of protocol/world pairs, $\mathcal{A}$ the abstract configuration space
 459 the checker explores, and $\text{abs} : \mathcal{C} \rightarrow \mathcal{A}$ the abstraction. We require:

$$(\text{step-sim}) \quad C \rightsquigarrow C' \Rightarrow \text{abs}(C) \rightsquigarrow^* \text{abs}(C'), \quad (\text{goal-sim}) \quad \text{goal}_{\varphi_{\text{goal}}}(C) \Rightarrow \hat{\text{goal}}(\text{abs}(C)).$$

460 **Lemma 1** (Transport). *If $C_0 \rightsquigarrow^* C$ then $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$.*

461 *Proof.* By induction on the reflexive-transitive closure. The empty run is reflexivity. For $C_0 \rightsquigarrow^*$
 462 $C' \rightsquigarrow C$, the induction hypothesis gives $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C')$ and STEP-SIM gives $\text{abs}(C') \rightsquigarrow^* \text{abs}(C)$;
 463 compose them. Mechanized as `reach_abs`, parameterized by STEP-SIM. $\square$

464 **Lemma 2** (Reachability monotonicity). *If every step of $\rightsquigarrow_1$ is a step of $\rightsquigarrow_2$ (that is, $x \rightsquigarrow_1 y$ implies
 465 $x \rightsquigarrow_2 y$), then $s \rightsquigarrow_1^* w$ implies $s \rightsquigarrow_2^* w$.*

466 *Proof.* By induction on the closure $s \rightsquigarrow_1^* w$. The empty run transports by reflexivity. For $s \rightsquigarrow_1^* u \rightsquigarrow_1$
 467 w , the induction hypothesis gives $s \rightsquigarrow_2^* u$; the hypothesis turns the last step $u \rightsquigarrow_1 w$ into $u \rightsquigarrow_2 w$;
 468 appending it gives $s \rightsquigarrow_2^* w$. Mechanized as `reach_mono`. $\square$

469 D.2 Refutation soundness, tolerance, monotonicity

470 **Theorem 1** (Refutation soundness). *If the abstract system has no reachable goal state from
 471 $\text{abs}(G, W_0)$, then no declared run of that pack configuration reaches φ_{goal} . Hence an impossi-
 472 ble verdict is never wrong relative to the pack, provided STEP-SIM and GOAL-SIM hold.*

473 *Proof.* We prove the contrapositive: if *some* declared run reaches the goal, then the abstract
 474 system reaches an abstract goal state. Suppose $C_0 \rightsquigarrow^* C$ with $\text{goal}_{\varphi_{\text{goal}}}(C)$. By Lemma 1,
 475 $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$, and by GOAL-SIM, $\hat{\text{goal}}(\text{abs}(C))$. Thus $\text{abs}(C)$ is an abstract goal state reach-
 476 able from $\text{abs}(C_0)$, contradicting the hypothesis. Hence no declared run reaches φ_{goal} : a verdict
 477 of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an
 478 abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the
 479 two simulation hypotheses (the existential witness is $\text{abs}(C)$): $\square$

```

480   Theorem refutation_sound : forall s0,
481     (~ exists a, reach_aste (abs s0) a /\ agoal a) ->
482     (~ exists w, reach_cstep s0 w /\ cgoal w).
483   (* Print Assumptions refutation_sound. => Closed under the global
484   context *)

```

485 **Theorem 2** (Tolerance soundness). *If a coarser over-approximation (more edges) cannot reach the
 486 goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false
 487 refutation.*

488 *Proof.* Let $\rightsquigarrow_{\text{fine}}$ and $\rightsquigarrow_{\text{coarse}}$ be the two abstract step relations, with the coarsening an *over-*
 489 approximation: every fine step is also a coarse step, $x \rightsquigarrow_{\text{fine}} y \Rightarrow x \rightsquigarrow_{\text{coarse}} y$. Suppose, for
 490 contradiction, that the finer system reaches the goal: $s_0 \rightsquigarrow_{\text{fine}}^* a$ with $\hat{goal}(a)$. By Lemma 2 (with
 491 $\rightsquigarrow_1 = \rightsquigarrow_{\text{fine}}$, $\rightsquigarrow_2 = \rightsquigarrow_{\text{coarse}}$), $s_0 \rightsquigarrow_{\text{coarse}}^* a$; and $\hat{goal}(a)$ still holds. So the coarser system reaches
 492 a goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer
 493 system: coarsening α only *adds* abstract edges, so it can never turn an unreachable goal into a
 494 reachable one — refutation is preserved. Mechanized as `tolerance_sound`, axiom-free. $\square$

495 **Theorem 3** (Capability monotonicity). *If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal} , then Γ' reaches φ_{goal} .*
 496 *Granting tools never makes an achievable goal impossible.*

497 *Proof.* Enlarging the capability context only *adds* guarded effects: since $\Gamma \subseteq \Gamma'$, every capability
 498 firing available under Γ is available under Γ' with the same guard and effect (WORLD-ACT in B.1
 499 quantifies over $a \in \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$), so every step of the Γ -transition relation is a step of
 500 the Γ' -relation. Suppose Γ reaches the goal configuration C from C_0 . By Lemma 2, the same
 501 configuration is reachable under Γ' , and $goal_{\varphi_{\text{goal}}}(C)$ is unchanged because the goal predicate does
 502 not depend on Γ . Hence Γ' reaches φ_{goal} via the same witnessing run. The other premises of ACH
 503 are monotone in the same direction ($\text{caps}(G) \subseteq \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$ still holds, and realizability
 504 and conformance do not mention Γ beyond $a \in \text{dom}(\Gamma)$), so the whole judgment is preserved. The
 505 transport half is mechanized as `cap_monotone`, axiom-free, stated over an inclusion of step relations;
 506 the reduction of $\Gamma \subseteq \Gamma'$ to that inclusion is the WORLD-ACT argument above, on paper. $\square$

507 **Concrete instance.** The flight skill of Section 2, variant A, is mechanized as `FlightInstance`. We
 508 prove `flight_refuted` (no run reaches `booked` $\wedge$ `confirmation_sent`, since no step touches `conf`)
 509 and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity
 510 of Theorem 1.

511 D.3 Operational correspondence: subject reduction and session fidelity

512 The direct judgment $G \vdash (\mathbb{M}; W)$ is not merely preserved by reduction: the session and its protocol
 513 step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness
 514 backbone of a session calculus [12, 11], obtained here *directly*, with no projection and no merge. For
 515 world-changing interleavings, the statements below assume two explicit frame conditions: effects of
 516 participant-disjoint actions commute, and an action moved under a pending marker preserves that
 517 marker's formula. Communication-only reductions satisfy both conditions vacuously.

518 **Lemma 3** (Residual capability). *If $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $\Lambda \in \text{cap}(G)$.*

519 **Lemma 4** (Participant agreement). *If $G \vdash (\mathbb{M}; W)$, then $\text{prt}(G) = \text{prt}(\mathbb{M})$.*

520 Both follow by induction on the transition or typing derivation, respectively; they are the capability-
 521 membership and participant invariants used in the interleaving cases below.

522 **Theorem 4** (Subject reduction). *If $G \vdash (\mathbb{M}; W)$ and $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$, then $(G, W) \rightsquigarrow_{\Lambda}$
 523 (G', W') for some G' with $G' \vdash (\mathbb{M}'; W')$.*

524 **Theorem 5** (Session fidelity). *If $G \vdash (\mathbb{M}; W)$ and $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $(\mathbb{M}, W) \xrightarrow{\Lambda}$
 525 $(\mathbb{M}', W')$ for some $\mathbb{M}'$ *and* W' with $G' \vdash (\mathbb{M}'; W')$.*

526 We prove both by induction on the typing derivation $G \vdash (\mathbb{M}; W)$, with a case analysis on the
 527 transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions
 528 (`ctypes_ext`), which lets the interleaving cases reassociate independent updates.

529 *Proof of Theorem 4 (subject reduction).* By induction on $G \vdash (\mathbb{M}; W)$.

530 *Case T-END* ($G = \text{End}$, every role idle). Then $\mathbb{M}$ has no output or pending action, so the only
 531 available transition is the goal observation S-GOAL at $\Lambda = \checkmark \varphi_{\text{goal}}$ when $W \models \varphi_{\text{goal}}$, which leaves
 532 $(\mathbb{M}, W)$ fixed; there is no residual type to preserve and the claim holds (otherwise no Λ -transition
 533 exists and it holds vacuously).

534 *Case T-GOAL* ($G = \checkmark \varphi.G_0$, with $W \models \varphi$ and $G_0 \vdash (\mathbb{M}; W)$, $\text{prt}(G_0) = \text{prt}(\mathbb{M})$). The transition
 535 $(\mathbb{M}, W) \rightarrow_{\Lambda} (\mathbb{M}', W')$ splits on its label.

536 • *Goal observation* ($\Lambda = \checkmark\varphi$, by S-GOAL, so $\mathbb{M}' = \mathbb{M}$ and $W' = W$). Since $W \models \varphi$, rule
 537 G-GOAL-E discharges the marker, $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\checkmark\varphi} (G_0, W)$, and $G_0 \vdash (\mathbb{M}; W)$ is the
 538 premise: take $G' = G_0$.

539 • *Interleaving* ($\Lambda \neq \checkmark\varphi$). A goal marker is not a session construct, so this is already a
 540 transition of G_0 's session; the induction hypothesis gives $(G_0, W) \rightsquigarrow_{\Lambda} (G', W')$ with
 541 $\Lambda \in \text{cap}(G_0)$ and $G' \vdash (\mathbb{M}'; W')$. When the move is world-preserving (a communica-
 542 tion, $W' = W$), G-GOAL-I carries the still-pending marker along, $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda}$
 543 $(\checkmark\varphi.G', W)$, and $\checkmark\varphi.G' \vdash (\mathbb{M}'; W)$ by T-GOAL (the side condition $\text{prt}(G') = \text{prt}(\mathbb{M}')$ is
 544 preserved because Λ acts on the same participants on both sides). When the move is *world-*
 545 *changing* (a capability firing, $W' \neq W$), G-GOAL-I still commutes it under the marker,
 546 $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda} (\checkmark\varphi.G', W')$; re-typing the residual by T-GOAL then additionally asks
 547 $W' \models \varphi$. This follows from the goal-stability hypothesis on actions commuting under pend-
 548 ing markers. The global type sequences the marker before G_0 's actions whereas the session
 549 may fire the action first; the interleaving rule lets the type catch up under that hypothesis.
 550 Removing the hypothesis, rather than this conditional case, is the open correspondence
 551 problem recorded in Section 7.

552 *Case T-COMM*, with head $p \rightarrow q$ and branches G_i . Here $\mathbb{M}$ is $p[q! \dots] \parallel q[p? \dots] \parallel \mathbb{M}_0$, and each
 553 branch satisfies $G_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0; W)$. If the transition is a goal observation $\Lambda = \checkmark\varphi$
 554 (S-GOAL, leaving $(\mathbb{M}, W)$ fixed), it is matched by G-COMM-I: $\text{prt}(\checkmark\varphi) = \emptyset$ makes the disjointness
 555 premise trivial, and each branch realises the same observation by the induction hypothesis, using
 556 $\checkmark\varphi \in \text{cap}(G_i)$ — the observation concerns a marker the protocol actually pends (see the note below).
 557 Otherwise the transition is a communication $\Lambda = r \ell t$. Because p offers only outputs to q and q only
 558 inputs from p , the sender/receiver shapes force a dichotomy.

559 • *Head move* ($r = p$). Then $t = q$ and $\ell = \ell_k \in I$, and $\mathbb{M}' = p[P_k] \parallel q[Q_k] \parallel \mathbb{M}_0$. Rule
 560 G-COMM-E gives $(G, W) \rightsquigarrow_{p\ell_k q} (G_k, W)$, and $G_k \vdash (\mathbb{M}'; W)$ is exactly the k -th premise.

561 • *Interleaving move* ($r \neq p$). The shapes then force $r, t \notin \{p, q\}$: r is an output so $r \neq q$,
 562 and t is an input so $t \neq p$, while $t = q$ would force $r = p$. Since r, t lie in the bystanders
 563 $\mathbb{M}_0$, the *same* communication is enabled in each branch's configuration $p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0$
 564 (the head updates do not touch r, t). The induction hypothesis at each i yields G'_i with
 565 $(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W)$ and $G'_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}'_0; W)$, where $\mathbb{M}'_0$ is $\mathbb{M}_0$ after the
 566 bystander step. As $\text{prt}(\Lambda) = \{r, t\}$ is disjoint from $\{p, q\}$, rule G-COMM-I assembles
 567 $(G, W) \rightsquigarrow_{\Lambda} (p \rightarrow q: \{\ell_i.G'_i\}, W)$, and re-applying T-COMM (the sender/receiver at p, q
 568 are untouched, and each branch is retyped by the induction hypothesis, reassociating the
 569 independent updates via `ctypes_ext`) gives the residual typing.

570 Communications leave the world fixed, so no independence hypothesis is needed. For a world-
 571 changing action at the head (T-ACT), G-ACT-E matches and the residual G is typed at the *same*
 572 post-effect world the rule reaches — this is exactly where the lockstep world pairing of the corrected
 573 T-ACT is used; a goal observation at an action-typed configuration is matched by G-ACT-I exactly
 574 as in the T-COMM case, since $\text{prt}(\checkmark\varphi) \cap \{p\} = \emptyset$ trivially. Interleaving *two* actions requires their
 575 effects to commute ($\text{eff}_a \circ \text{eff}_b = \text{eff}_b \circ \text{eff}_a$ for disjoint a, b), a frame-independence side condition
 576 that participant-disjointness alone does not supply; under it the T-ACT interleaving case goes through
 577 identically. $\square$

578 *Proof of Theorem 5 (session fidelity)*. By induction on $G \vdash (\mathbb{M}; W)$, inverting the global step
 579 $(G, W) \rightsquigarrow_{\Lambda} (G', W')$. T-END admits only the goal observation G-GOAL-E at $\checkmark\varphi_{\text{goal}}$, matched
 580 on the session by S-GOAL. For T-GOAL, the step is either G-GOAL-E — matched by S-GOAL
 581 on the session, the marker discharged — or G-GOAL-I, whose premise $(G_0, W) \rightsquigarrow_{\Lambda} (G'_0, W')$
 582 the induction hypothesis realises as an underlying session step that carries the marker along. For
 583 T-COMM, the global step is either G-COMM-E — and the prescribed head communication $p \ell_k q$ is
 584 enabled by S-COMM, landing in the k -th premise — or G-COMM-I with $\text{prt}(\Lambda) \cap \{p, q\} = \emptyset$; then
 585 the induction hypothesis on any branch produces a bystander communication, whose endpoints lie
 586 outside $\{p, q\}$, so S-COMM fires it on $\mathbb{M}$ and T-COMM retypes the result. The action cases mirror
 587 those of Theorem 4. $\square$

588 **Mechanization.** Both theorems are mechanized axiom-free for the single-label communication
 589 fragment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`,
 590 over labelled session and global transition systems, with no projection, merge, or local type:

```

591 Theorem subject_reduction : forall W G s s' L,
592   ctypes W G s -> lstep L s s' ->
593   exists G', gstep W L G G' /\ ctypes W G' s'.
594 Theorem session_fidelity : forall W G G' s L,
595   ctypes W G s -> gstep W L G G' ->
596   exists s', lstep L s s' /\ ctypes W G' s'.
597 (* Print Assumptions => Closed under the global context (both). *)

```

598 The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v`
 599 (`type_directed_safety`, also `progress`). Observable goal labels appear in neither Coq develop-
 600 ment: the mechanized label alphabet is communication-only, and its goal rule fuses marker discharge
 601 with a continuation step. Participant-matching side conditions, labelled action interleaving, and
 602 multi-branch choice are likewise absent. Completing a faithful mechanization of the paper rules
 603 remains work in Section 7.

604 **Concrete instance.** `HandoffInstance` mechanizes the planner/worker handoff of Section 1 on both
 605 sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed
 606 (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The
 607 *bad* pairing — both roles start with an input, exactly the deadlock of Section 1 — is proved `Stuck`
 608 (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type*
 609 *can ever type it* (`bad_untypable`): the rule rejects the classical deadlocking handoff without any
 610 projection ever being computed.

611 D.4 Incompleteness, by design

612 **Proposition 1** (Incompleteness). $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ *may hold while no concrete runtime execution*
 613 *reaches φ_{goal} : the witnessing abstract path can be spurious when α collapses a distinction that*
 614 *mattered. The system is sound for $\times$ and incomplete for $\checkmark$.*

615 This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision.
 616 The residue is confined to two edges, motivating the layered architecture:

- 617 • **Payload faithfulness** (bottom edge): a tool’s declared post-condition (“filters to fares
 618 < 500”) may be false — a runtime obligation for a deterministic monitor (Layer C), not a
 619 static one.
- 620 • **Intent fidelity** (top edge): the formalized goal may not capture what the user meant —
 621 irreducibly semantic, surfaced at the single compaction checkpoint for human review.

622 D.5 Decidability and the autonomy boundary

623 **Theorem 6** (Decidability). *In the fragment with finitely many roles (static topology), regular global*
 624 *types represented by finite tail-recursive control graphs, decidable state logic $\mathcal{L}$, and $\mathcal{L}$ -definable*
 625 *effects, $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ is decidable.*

626 *Proof.* The pack-level achievability judgment $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ (Section B.2) is an existential
 627 reachability query over configurations (G', W) ; we exhibit a terminating decision procedure and
 628 argue its completeness.

629 *The search space is finite.* A configuration has two components. (i) *Control*, the residual global
 630 type G' reachable from G by $\rightsquigarrow$. Since G is a regular tree (tail-recursion: finitely many distinct
 631 subtrees, Section A.3) and each $\rightsquigarrow$ step either descends into a subtree (G-COMM-E, G-ACT-E) or
 632 erases a marker (G-GOAL-E), the set $\{G' : (G, _) \rightsquigarrow^* (G', _)\}$ is a subset of the finitely many
 633 subtrees of G , hence finite. (ii) *Symbolic world*. The boolean part is a valuation $B : \text{Pred} \rightarrow \mathbb{B}$
 634 over the finitely many predicates named in the pack, so there are $2^{|\text{Pred}|}$ boolean states. The numeric
 635 part is summarized by an accumulated constraint $\psi \in \mathcal{L}$ (a conjunction of the guards taken and
 636 the Nd -postconditions assumed); on a control-graph back edge the reference procedure applies the
 637 widening $\psi \mapsto \top$, forgetting accumulated numeric constraints. This one-element numeric summary

is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

Each edge is computable. Firing a capability requires checking $W \models pre_a$ and forming sp ; testing goal-satisfaction requires $W \models \varphi$. Both are entailment/satisfiability queries in quantifier-free linear integer arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over *init*; each satisfying symbolic initial valuation seeds the finite search.

Termination and correctness. Forward search (breadth-first over $\rightsquigarrow$) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the procedure is a decision procedure for $\Gamma; G \models \Diamond_{init} \varphi_{goal}$. $\square$

Proposition 2 (Undecidability in a dynamic-topology extension). *Consider the extension G^+ of the grammar with spawn $r.G$, which creates fresh participants at run time. With an unbounded role population and dynamic channels, achievability in G^+ is undecidable.*

Proof sketch. By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [14]. Fix a CFSM instance: finite-control machines M_1, M_2 exchanging messages over unbounded FIFO channels, and a target control state q_f of M_1 ; deciding whether a run reaches q_f is undecidable.

We compute, from this instance, a skill whose goal is achievable iff the CFSM reaches q_f . Each machine M_i becomes a role whose process mirrors its control graph: a transition that sends message m spawns, at run time, a fresh *courier* participant carrying m (this is exactly the dynamic-spawning capability the theorem hypothesizes), and a transition that receives m synchronizes with the oldest outstanding courier for that channel, so the unbounded family of couriers realizes the unbounded FIFO. The required order can be made explicit by assigning sequence numbers through the numeric world state and guarding receives on the next sequence number; the *Asg/Nd* machinery of Section A.2 supplies those updates. A boolean predicate at_q_f is established by the single capability guarding M_1 's entry into q_f , and we set $\varphi_{goal} = at_q_f$. By construction a run of the skill reaches a state satisfying φ_{goal} iff the CFSM has a run reaching q_f : the courier discipline then puts the skill's reachable configurations in correspondence with the CFSM's channel contents and control states. The construction is a computable map from CFSM instances to packs, so a decision procedure for achievability would decide CFSM control-state reachability — impossible. Hence achievability is undecidable once participants may be spawned dynamically. (The finiteness argument of Theorem 6 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of reachable controls infinite.) $\square$

The two results isolate one concrete boundary: static finite topology admits a total decision procedure, while unbounded dynamic spawning does not. This is a boundary on topology, not a claim that all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a pack violates the static-topology side condition of Theorem 6, unless an independent capability or establisher refutation has already proved it impossible.

D.6 The partition of obligations

Corollary 1. *The architecture separates “will the goal be achieved” into four obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, decidable	Coq specification; deterministic search; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\square$ + permission tiers
payload faithfulness	Layer C (future), run-time	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

D.7 Verifiable feedback beyond a scalar reward

The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the decidable fragment, not a refutation. These structured outcomes can serve as cheap negative labels for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint keeps the irreducible semantic decision with a human. We treat such search as an application of the verifier, not as a contribution evaluated here.

E Token economics of pre-execution refutation

The practical case for deciding achievability *before* execution is an economic one, so we state it in tokens rather than adjectives. Two asymmetries carry it. First, no model sits in the trusted path: the schema gate, projection, conformance and Z3 reachability spend **zero tokens**, and the deterministic front-end spends zero as well. Only the optional LLM compaction spends tokens, and it is paid *once per skill version*. Second, an agent turn re-sends its whole context: writing S for the harness prompt, K for the loaded skill, g for the mean tokens appended per turn and o for output per turn, a run of T turns reads $T(S+K) + gT(T-1)/2$ input and To output tokens. Compaction is linear in the skill and paid once; a doomed run is *quadratic* in its turns and recurs on every invocation.

Runtime waste is necessarily a model — it prices a run that, if the refutation is correct, never happens — so we publish the model rather than a single number. Each refutation reason carries a (low, typical, high) band for how long an agent flails before that failure stops it, and how many agents are billed. With conservative defaults ($S=2500$, $g=700$, $o=250$) and a median real consumer skill ($K \approx 1.7K$ tokens), compaction costs $\sim 2.8K$ tokens against the following avoided waste per prevented invocation:

Verdict reason	turns (lo–typ–hi)	waste/run	leverage
MISSING_CAPABILITY	3–8–14, 1 agent	50 240	18×
GOAL_UNSAT	8–18–30, 1 agent	176 040	63×
NON_CONFORMANT	6–15–30, 2 agents	261 900	94×
NON_PROJECTABLE	6–15–30, 2 agents	261 900	94×
BLOCKED_GUARD	12–25–50, 1 agent	305 750	110×

The ordering is the robust part, and it is the expected one. MISSING_CAPABILITY is cheapest at run time: the agent calls a tool that is not there and learns immediately, and most of the cost is the improvisation that follows. BLOCKED_GUARD is dearest, because a precondition no run can satisfy is precisely the case an agent cannot learn from — it retries to the harness turn cap. The two multi-party reasons bill two contexts. GOAL_UNSAT additionally carries a cost the token bill does not show: a run whose goal conjunct has no establisher can terminate *believing* it succeeded. DYNAMIC_TOPOLOGY claims no savings at all, because an abstention prevents nothing.

Over the 15-spec corpus the seven structural refutations cost 12.2K tokens of compaction and avoid $\sim 1.07M$ tokens per round of invocations (band 0.28M–3.15M): $\sim 87\times$ leverage, or nothing at all on the deterministic path. Break-even therefore falls *inside* the first prevented run in every mode — between 0.9% and 5.5% of it. The honest denominator for a healthy skill, where the check buys nothing and still costs something, is 2.9% of one successful run over the corpus and 4.0% for a median real skill; with the deterministic front-end, zero. Prompt caching changes the price of the wasted tokens, not their number. Halving every waste default leaves leverage at $9\times$ – $55\times$ and the ordering unchanged. The model, its defaults and their justification are in the artifact (skillc.tokens, skillc.cost); the Coq development costs no tokens and is amortized over every skill ever checked.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

727 Answer: [Yes]
728 Justification: The abstract and Section 1 state that refutation is sound relative to the declared
729 pack, distinguish UNKNOWN from refutation, and identify the mechanized and paper-only
730 proof fragments.

731 **2. Limitations**
732 Question: Does the paper discuss the limitations of the work performed by the authors?
733 Answer: [Yes]
734 Justification: The Limitations and Future Work section discusses relative soundness, static
735 topology, abstraction coarseness, mechanization gaps, and the proof-of-concept corpus.

736 **3. Theory assumptions and proofs**
737 Question: For each theoretical result, does the paper provide the full set of assumptions and
738 a complete (and correct) proof?
739 Answer: [Yes]
740 Justification: Appendix D states the simulation, stability, commutativity, finiteness, and
741 topology assumptions with proofs or proof sketches; the artifact checks the explicitly
742 identified Coq fragments.

743 **4. Experimental result reproducibility**
744 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
745 perimental results of the paper to the extent that it affects the main claims and/or conclusions
746 of the paper (regardless of whether the code and data are provided or not)?
747 Answer: [Yes]
748 Justification: Sections 4–5 describe the deterministic checker, verdict semantics, corpus
749 construction, binary accounting, and separate treatment of abstentions.

750 **5. Open access to data and code**
751 Question: Does the paper provide open access to the data and code, with sufficient instruc-
752 tions to faithfully reproduce the main experimental results, as described in supplemental
753 material?
754 Answer: [Yes]
755 Justification: The anonymized supplemental artifact contains the checker, synthetic corpora,
756 proof developments, pinned continuous-integration workflow, and reproduction commands.

757 **6. Experimental setting/details**
758 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
759 rameters, how they were chosen, type of optimizer) necessary to understand the results?
760 Answer: [N/A]
761 Justification: The evaluation is a deterministic execution over declared packs; it performs no
762 training, optimization, sampling, or data splitting.

763 **7. Experiment statistical significance**
764 Question: Does the paper report error bars suitably and correctly defined or other appropriate
765 information about the statistical significance of the experiments?
766 Answer: [N/A]
767 Justification: Each corpus case and checker result is deterministic, so repeated trials have no
768 sampling variance; the paper reports exact counts rather than inferential statistics.

769 **8. Experiments compute resources**
770 Question: For each experiment, does the paper provide sufficient information on the com-
771 puter resources (type of compute workers, memory, time of execution) needed to reproduce
772 the experiments?
773 Answer: [Yes]
774 Justification: Section 5 states that evaluation is CPU-only, requires no model inference or
775 training, and gives the corpus size and solver timeout governing the run.

776	9. Code of ethics
777	Question: Does the research conducted in the paper conform, in every respect, with the
778	NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?
779	Answer: [Yes]
780	Justification: The work uses synthetic packs, no personal data or human subjects, and makes
781	bounded claims with explicit trust assumptions and limitations.
782	10. Broader impacts
783	Question: Does the paper discuss both potential positive societal impacts and negative
784	societal impacts of the work performed?
785	Answer: [Yes]
786	Justification: The Broader impact paragraph discusses reduced wasted execution as well as
787	false confidence from inaccurate packs and the human/runtime mitigations required.
788	11. Safeguards
789	Question: Does the paper describe safeguards that have been put in place for responsible
790	release of data or models that have a high risk for misuse (e.g., pre-trained language models,
791	image generators, or scraped datasets)?
792	Answer: [N/A]
793	Justification: The artifact releases no trained model, scraped dataset, or other high-risk asset;
794	it contains a deterministic checker and synthetic examples.
795	12. Licenses for existing assets
796	Question: Are the creators or original owners of assets (e.g., code, data, models), used in
797	the paper, properly credited and are the license and terms of use explicitly mentioned and
798	properly respected?
799	Answer: [N/A]
800	Justification: The evaluation does not reuse an external dataset or model; prior methods and
801	software foundations are credited through citations.
802	13. New assets
803	Question: Are new assets introduced in the paper well documented and is the documentation
804	provided alongside the assets?
805	Answer: [Yes]
806	Justification: The supplemental artifact documents the checker interface, pack schema,
807	synthetic corpora, proof scope, limitations, and reproduction workflow.
808	14. Crowdsourcing and research with human subjects
809	Question: For crowdsourcing experiments and research with human subjects, does the paper
810	include the full text of instructions given to participants and screenshots, if applicable, as
811	well as details about compensation (if any)?
812	Answer: [N/A]
813	Justification: The research involves neither crowdsourcing nor human-subject experiments.
814	15. Institutional review board (IRB) approvals or equivalent for research with human
815	subjects
816	Question: Does the paper describe potential risks incurred by study participants, whether
817	such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
818	approvals (or an equivalent approval/review based on the requirements of your country or
819	institution) were obtained?
820	Answer: [N/A]
821	Justification: The research involves no study participants or human-subject data.
822	16. Declaration of LLM usage

823 Question: Does the paper describe the usage of LLMs if it is an important, original, or
824 non-standard component of the core methods in this research? Note that if the LLM is used
825 only for writing, editing, or formatting purposes and does not impact the core methodology,
826 scientific rigor, or originality of the research, declaration is not required.

827 Answer: [\[Yes\]](#)

828 Justification: Sections 1 and 4 describe LLM compaction and bounded repair, explicitly
829 place them outside the trusted checker, and state that the reported corpus evaluation does
830 not depend on live model inference.